Gtkdialog User's Manual

Incomplete introduction

Laszlo Pere

This manual documents version 0.9.0 of the Gtkdialog utility.

WARNING: This documentation is very incomplete and has not been updated since at least 2007 although it is still informative for beginners.

Table of Contents

1 Getting Gtkdialog

1.1 Download

The source of Gtkdialog can be downloaded from the following URL `https://github.com/laszlopere/gtkdialog3`.

1.2 How to Install the Program

The program can be installed using the standard `./configure`, `make` and `make install` command sequence. Further details can be found in the `INSTALL` file included.

1.3 Copying

2 Introduction

Gtkdialog is a small utility program based on the GTK library. The program is mainly made for GUI development for shell scripts but can be used with many other programming languages. The programmer can easily create a GUI not just for any shell script or UNIX command but for any interpreted or compiled program capable of starting a child process and using pipes.

3 The Description Language

Gtkdialog3 uses an XML-like markup language to describe dialog boxes. The language looks like XML and follows its basic structure — tags with attributes, nested elements, and self-closing tags — but it is not standard XML. The parser recognises a fixed set of tag names (see Chapter 5 [Widgets], page 15, see Chapter 6 [Containers], page 29) and has several non-standard extensions described below.

All of the non-standard language elements listed in this chapter are **deprecated** and will be removed in a future version. New dialog descriptions should use only well-formed XML constructs.

3.1 Backslash escaping

Standard XML uses entity references (`<`, `>`, `&`) to represent special characters. Gtkdialog3 does not support entity references. Instead, it uses backslash escaping:

`\<` A literal `<` character (prevents it from being parsed as a tag).

`\>` A literal `>` character.

`\"` A literal double-quote inside a quoted string.

`\\` A literal backslash.

Example — displaying the text '`<button>`' as a label:

```
<text><label>\<button\></label></text>
```

In standard XML, this would be written as:

```
<text><label>&lt;button&gt;</label></text>
```

3.2 Space in tag names

XML tag names cannot contain spaces. Earlier versions of gtkdialog3 used `<input file>` and `<output file>` as tag names with an embedded space. Both forms have been removed; use the `type` attribute instead.

The `<input>` sub-element accepts a `type` attribute with the following values:

`type="bash"`
: Execute the content as a shell command (same as plain `<input>`).

`type="file"`
: Read the widget's value from the file named in the content.

`type="stock"`
: Load a GTK stock icon. The content is the stock icon ID (e.g. `gtk-open`).

`type="icon"`
: Load a GTK theme icon. The content is the icon name.

```
<input type="bash">ls /usr/bin</input>
<input type="file">/path/to/file</input>
<input type="stock">gtk-open</input>
<input type="icon">document-open</input>
```

3.3 Imperative constructs

The language includes simple imperative programming constructs that can be placed alongside widget elements. These are not part of XML and exist for dynamic dialog generation at parse time.

`<command assignment="var := expr">`
: Assigns a value to a variable. The expression supports arithmetic operators (`+`, `-`, `*`, `/`) and numeric constants.

`<if condition="expr">...</if>`
: Conditionally includes the enclosed widgets. The condition supports variable names, numbers, and the comparison operators `=` and `!=`.

`<while condition="expr">...</while>`
: Repeatedly includes the enclosed widgets as long as the condition is true.

`<show/>` Forces all widgets created so far to be displayed immediately.

Example — creating five buttons in a loop:

```
<command assignment="i := 0">
<while condition="i != 5">
    <button><label>Button</label></button>
    <command assignment="i := i + 1">
</while>
```

3.4 Implicit window element

Standard XML requires a single root element. In gtkdialog3, the outermost `<window>` tag can be omitted; the parser wraps the content in an implicit window automatically. This means a dialog can start directly with a container:

```
<vbox>
    <text><label>Hello</label></text>
    <button stock="ok"/>
</vbox>
```

This is equivalent to:

```
<window>
    <vbox>
        <text><label>Hello</label></text>
        <button stock="ok"/>
    </vbox>
</window>
```

New dialog descriptions should always include an explicit `<window>` element.

3.5 Unsupported XML features

The following standard XML features are **not supported** by gtkdialog3's parser:

- XML declarations (`<?xml version="1.0"?>`)
- Document type definitions (`<!DOCTYPE ...>`)

- CDATA sections (`<![CDATA[...]]>`)
- Entity references (`&amp;`, `&lt;`, `&gt;`, `&quot;`, `&apos;`)
- Processing instructions (`<?...?>`)
- XML comments (`<!-- ... -->`)
- Namespaces

Including any of these in a dialog description will cause a parse error.

4 Invoking the Program

Gtkdialog3 reads a dialog description from one of several sources, displays the resulting window, and prints the widget values to standard output when the dialog closes.

4.1 Command-line options

`-v, --version`
: Print version information (including which optional libraries are enabled) and exit.

`-d, --debug`
: Debug mode; prints the processed characters as gtkdialog3 parses the input.

`-p, --program=`*`VARIABLE`*
: Read the dialog description from the named environment variable.

`-f, --file=`*`FILENAME`*
: Read the dialog description from a file.

`-s, --stdin`
: Read the dialog description from standard input.

`-e, --event-driven=`*`FILENAME`*
: Execute the file as an event-driven program.

`-u, --ui-file=`*`FILENAME`*
: Read the dialog description from a GtkBuilder UI file.

`-i, --include=`*`FILENAME`*
: Source the given shell script before executing action commands, making its functions and variables available to `<action>` handlers.

`-w, --no-warning`
: Suppress warning messages.

`-G, --geometry=`*`[WxH][+X+Y]`*
: Set the size and placement of the window. W and H are the width and height in pixels; X and Y are the screen coordinates.

`-c, --center`
: Center the window on the screen.

`--space-expand=`*`STATE`*
: Set the default "expand" packing for all widgets (`true` or `false`).

`--space-fill=`*`STATE`*
: Set the default "fill" packing for all widgets (`true` or `false`).

`--output-format=`*`FORMAT`*
: Set the output format: `bash` (default), `json`, or `xml`.

`--print-ir`
: Print the internal representation of the parsed program and exit without displaying a window. Useful for debugging the dialog description.

`--debug-geometry`
: Print widget geometry negotiations to stderr, showing preferred and allocated sizes for each widget. Can also be enabled by setting the environment variable `GTKDIALOG_DEBUG_GEOMETRY`.

4.2 Reading from an environment variable

The most common way to use gtkdialog3. Store the dialog description in a shell variable, export it, and pass its name with `--program`:

From `examples/standard/entry`:

```
#!/bin/sh

MAIN_DIALOG="
<window title='System Configuration' resizable='false'>
    <vbox spacing='6' border-width='8'>
        <frame title='Hostname'>
            <entry>
                <default>localhost</default>
                <variable>entHostname</variable>
            </entry>
        </frame>
        <buttonbox layout='end'>
            <button stock='cancel'/>
            <button stock='ok'/>
        </buttonbox>
    </vbox>
</window>"
export MAIN_DIALOG

gtkdialog3 --program=MAIN_DIALOG
```

4.3 Capturing widget values

When the dialog closes, gtkdialog3 prints one *`VARIABLE`*`="value"` assignment per line to standard output, plus an `EXIT` variable indicating which button was pressed. Evaluate the output to import the values into the calling script:

```
I=$IFS; IFS=""
for STATEMENTS in $(gtkdialog3 --program=DIALOG); do
    eval $STATEMENTS
done
IFS=$I

if [ "$EXIT" = "OK" ]; then
    echo "You entered: $ENTRY."
else
    echo "You pressed Cancel."
fi
```

Changing `IFS` to the empty string protects spaces in user input from being split by the shell's word splitting.

4.4 Output formats

By default, gtkdialog3 prints variable assignments in Bash syntax. Two alternative formats are available via `--output-format`:

4.4.1 JSON output

```
gtkdialog3 --program=MAIN_DIALOG --output-format=json
```

Produces a JSON object grouped by window name:

```
{
  "widget_contents": {
    "myWindow": {
      "entName": "Alice",
      "chkAgreed": "true"
    }
  },
  "exit": "OK"
}
```

The window name is taken from the `<variable>` element inside each `<window>` tag (or auto-generated if none is specified).

4.4.2 XML output

```
gtkdialog3 --program=MAIN_DIALOG --output-format=xml
```

Produces an XML document with the same grouping by window name:

```
<?xml version="1.0" encoding="UTF-8"?>
<gtkdialog>
  <widget_contents>
    <window name="myWindow">
      <variable name="entName">Alice</variable>
      <variable name="chkAgreed">true</variable>
    </window>
  </widget_contents>
  <exit>OK</exit>
</gtkdialog>
```

4.5 Reading from a file

Use `--file` to load the dialog description from a file:

```
gtkdialog3 --file=dialog.xml
```

A file containing only the dialog description can also be made directly executable with a shebang line:

From `examples/languages/standalone_gtkdialog3_script`:

```
#! /usr/local/bin/gtkdialog3 -f
```

```
<window>
    <vbox>
        <checkbox>
            <label>This is a checkbox...</label>
            <variable>CHECKBOX</variable>
        </checkbox>
        <hbox>
            <button stock="ok"/>
            <button stock="cancel"/>
        </hbox>
    </vbox>
</window>
```

When used as a standalone script, widget values are printed to the standard output of the script as usual.

4.6 Reading from standard input

Use `--stdin` to read the dialog description from a pipe:

```
echo '<window><vbox><button stock="ok"/></vbox></window>' \
    | gtkdialog3 --stdin
```

This is useful for generating dialog descriptions dynamically in a pipeline.

4.7 Including function libraries

Use `--include` to source a shell script before any action command runs. This makes the script's functions and variables available to `<action>` handlers.

```
gtkdialog3 --include=functions.sh --program=MAIN_DIALOG
```

Without `--include`, action commands run in a fresh subshell that only has the exported environment variables. With it, the included file is dot-sourced (via `. file`) before each action command, so unexported shell functions become available.

4.8 GtkBuilder UI files

Instead of writing the dialog in gtkdialog3's own XML language, you can design the interface in a visual tool such as Glade and load the resulting GtkBuilder UI file with `--ui-file`. Gtkdialog3 registers every widget from the UI file, exports their values on exit (just like native dialogs), and connects signals so that the `handler` attribute of each `<signal>` element is executed as a shell command.

4.8.1 Basic usage

```
gtkdialog3 --ui-file=dialog.ui --program=main_window
```

The `--program` argument selects which top-level window to show. It must match an `id` attribute in the UI file. If omitted, gtkdialog3 looks for a widget called `MAIN_WINDOW`.

4.8.2 Signal handlers

In a GtkBuilder UI file the `handler` attribute of a `<signal>` element is normally a C function name. When loaded by gtkdialog3, the handler string is instead executed as a shell command, exactly like an `<action>` element in the native language. The same action prefixes are supported:

`handler="`*`command`*`"`
: Run *command* in a shell. Widget values are available as `$`*`id`* variables.

`handler="exit:`*`label`*`"`
: Close the dialog with the given exit label.

`handler="refresh:`*`id`*`"`
: Re-read the `<input>` of the named widget.

Multiple `<signal>` elements on the same widget are supported; they execute in order.

A `realize` signal is handled specially: its handler string is stored as the widget's `<input>` and processed when the widget is first shown, which lets you populate fields at startup. The handler string supports the same prefixes as the native `<input>` element:

`handler="`*`command`*`"`
: Run *command* in a shell and use its output to populate the widget (the `Command:` prefix is added automatically).

`handler="command:`*`command`*`"`
: Same as above, with an explicit prefix.

`handler="file:`*`/path/to/file`*`"`
: Read the widget's initial value from a file.

4.8.3 Including shell functions

Use `--include` together with `--ui-file` to make shell functions available to signal handlers:

```
gtkdialog3 --ui-file=dialog.ui \
           --include=functions.sh \
           --program=main_window
```

4.8.4 Designing with Glade

The recommended way to create UI files is the Glade interface designer (`glade`, available as the `glade` package on Debian/Ubuntu). Design the layout visually, then save and load the file with `--ui-file`. Modern versions of Glade (3.x and later) always save in GtkBuilder format regardless of whether the file extension is `.glade` or `.ui` — both work identically with gtkdialog3.

Setting widget IDs

Every widget that you want gtkdialog3 to export must have an **ID** set in Glade (the first field at the top of the General properties tab). The ID becomes the shell variable name in the output. Widgets without an ID receive auto-generated names such as `___object_1___`, which are not useful in scripts.

The top-level window *must* have an ID. By default gtkdialog3 looks for `MAIN_WINDOW`; pass `--program=`*`id`* to select a different one.

Signal handlers in Glade

Glade's signal editor expects C function names in the `Handler` column, but the `TYPE:name` action format used by gtkdialog3 is accepted without problems because it contains no spaces or special characters. Actions such as `exit:OK`, `exit:Cancel`, `refresh:myEntry`, or `closewindow:myDialog` can be typed directly into the Handler field. Shell commands that contain spaces or quotes are harder to enter in Glade; for those, define a shell function in a separate file, load it with `--include`, and use the function name as the handler.

Default values and dynamic initialisation

Static default values (such as the `text` property of a GtkEntry) can be set in Glade's property editor and are applied automatically. For dynamic initialisation at startup, add a `realize` signal whose handler is the command to run — Glade supports this in its signal editor as well.

4.8.5 Limitations compared to native dialogs

GtkBuilder UI files are designed for layout, not for gtkdialog3's data-binding model. The following native XML features have no direct equivalent in the GtkBuilder format:

- `<item>` tags — populating comboboxes, lists, or trees with rows of data.
- `<output file="/path">` — writing the widget value to a file whenever it changes.

Shell-command and file-based input (`<input>`) can be achieved via `realize` signal handlers (see Section 4.8 [GtkBuilder UI files], page 9). For dialogs that need item lists or file output, use gtkdialog3's native XML language instead.

4.8.6 UI file format

The UI file is a standard GtkBuilder XML document. A minimal example with an entry field and two buttons:

```
<?xml version="1.0" encoding="UTF-8"?>
<interface>
  <object class="GtkWindow" id="main_window">
    <property name="title">Example</property>
    <child>
      <object class="GtkBox" id="vbox">
        <property name="visible">True</property>
        <property name="orientation">vertical</property>
        <child>
          <object class="GtkEntry" id="name_entry">
            <property name="visible">True</property>
          </object>
        </child>
        <child>
          <object class="GtkButton" id="ok_button">
            <property name="visible">True</property>
            <property name="label">OK</property>
            <signal name="clicked" handler="exit:OK"/>
          </object>
```

```
          </child>
          <child>
            <object class="GtkButton" id="cancel_button">
              <property name="visible">True</property>
              <property name="label">Cancel</property>
              <signal name="clicked" handler="exit:Cancel"/>
            </object>
          </child>
        </object>
      </child>
    </object>
  </interface>
```

Each `<object>` element corresponds to a GTK widget. The `id` attribute becomes the variable name that gtkdialog3 exports on exit. Properties, packing, and child relationships follow the standard GtkBuilder schema; see the GtkBuilder documentation (`https://docs.gtk.org/gtk3/class.Builder.html`) for the full reference.

4.8.7 Supported widget types

Gtkdialog3 recognises the following GTK widget classes in UI files and maps them to its internal types for value export and signal handling:

GtkBuilder class	**Gtkdialog3 type**
GtkWindow	window
GtkButton	button
GtkToggleButton	togglebutton
GtkCheckButton	checkbox
GtkRadioButton	radiobutton
GtkColorButton	colorbutton
GtkFontButton	fontbutton
GtkEntry	entry
GtkSpinButton	spinbutton
GtkComboBox	comboboxtext
GtkComboBox (has-entry)	comboboxentry
GtkScale (horizontal)	hscale
GtkScale (vertical)	vscale
GtkProgressBar	progressbar
GtkLabel	text
GtkImage	pixmap
GtkTextView	edit
GtkTreeView	tree
GtkNotebook	notebook
GtkFrame	frame
GtkExpander	expander
GtkBox (horizontal)	hbox
GtkBox (vertical)	vbox
GtkMenuBar	menubar

GtkMenu	menu
GtkMenuItem	menuitem
GtkSeparatorMenuItem	menuitemseparator
GtkStatusbar	statusbar
GtkEventBox	eventbox
GtkScrolledWindow	scrolledwindow
GtkFileChooserWidget	chooser
GtkSeparator (horizontal)	hseparator
GtkSeparator (vertical)	vseparator

Widgets not listed above are still loaded and displayed, but their values are not exported and their signals must be generic GtkWidget signals.

4.9 Multiple windows

Define each window in a separate exported shell variable. The variable name must match the `<variable>` directive inside the `<window>` element. Windows can open and close each other using the `launch:` and `closewindow:` actions (see Section 7.9.3 [launch], page 36, see Section 7.9.4 [closewindow], page 36).

From `examples/languages/bash_script_multi_window`:

```
winMain='
<window title="winMain" width-request="250">
    <vbox>
        <button>
            <label>Open second window</label>
            <action function="launch">winLaunch1</action>
        </button>
        <button stock="cancel"/>
    </vbox>
    <variable>winMain</variable>
</window>'
export winMain

winLaunch1='
<window title="winLaunch1" width-request="250">
    <vbox>
        <text><label>Second window</label></text>
        <button>
            <label>Close this window</label>
            <action function="closewindow">winLaunch1</action>
        </button>
    </vbox>
    <variable>winLaunch1</variable>
</window>'
export winLaunch1

gtkdialog3 --program=winMain
```

4.10 Layout options

The `--space-expand` and `--space-fill` flags set the default packing behaviour for all widgets in the dialog. These defaults can be overridden per widget using the `space-expand` and `space-fill` tag attributes.

```
gtkdialog3 --space-expand=true --space-fill=true \
    --program=MAIN_DIALOG
```

When `space-expand` is `true`, widgets request extra space from their parent container. When `space-fill` is `true`, widgets fill the extra space they are given. Setting both to `true` is common for dialogs where widgets should stretch to fill the window.

4.11 Window placement

Use `--center` to center the window on the screen:

```
gtkdialog3 --center --program=MAIN_DIALOG
```

Use `--geometry` for precise control over window size and position. The format is *WxH+X+Y*, where W/H are the width and height in pixels and X/Y are the screen coordinates:

```
gtkdialog3 --geometry=400x300+100+200 --program=MAIN_DIALOG
```

5 Widgets

The dialog description language is a simple XML-like markup capable of describing complex dialog boxes containing widgets and containers.

Widgets are GUI elements such as buttons, entry fields, lists, and labels. Widgets are configured by placing sub-elements between their opening and closing tags (see Chapter 7 [Widget Sub-elements], page 32). Widgets must be placed inside containers (see Chapter 6 [Containers], page 29).

5.1 Tag attributes and GTK properties

Opening tags can carry inline attributes that control the widget's appearance and behaviour:

```
<entry visible="false" width-request="300">
<window title="My App" resizable="false">
<vbox spacing="10" border-width="8">
```

Any attribute whose name matches a GObject property of the underlying GTK widget is applied directly. This means you can use any standard GTK property without gtkdialog3 needing explicit support for it. Boolean properties accept `true`/`false`, integer and float properties accept numeric values, and enum properties accept the GLib nick string (e.g. `start`, `center`, `end`, `fill`).

Some commonly used GTK widget properties:

`halign`, `valign`
: Horizontal/vertical alignment: `fill`, `start`, `end`, `center`, `baseline`.

`hexpand`, `vexpand`
: Whether the widget requests extra space (`true`/`false`).

`margin-start`, `margin-end`, `margin-top`, `margin-bottom`
: Margins in pixels.

`width-request`, `height-request`
: Minimum widget size in pixels.

`sensitive`
: Whether the widget responds to input (`true`/`false`).

`visible`
: Whether the widget is shown (`true`/`false`).

The full list of properties for each widget can be found in the GTK 3 documentation at `https://docs.gtk.org/gtk3/`. Look up the widget class (e.g. GtkEntry, GtkButton, GtkBox) and check its "Properties" section. Container-specific properties such as `spacing`, `homogeneous`, and `border-width` are documented with their respective container widgets (see Chapter 6 [Containers], page 29).

5.2 `<text>` — Static label

A non-editable text label. The text can be set with `<label>`, loaded from a file with `<input type="file">`, or generated by a shell command with `<input>`. Refreshing the widget re-evaluates its input source, so labels can show dynamic content.

Exports its displayed text as the variable value.

From `examples/text/text`:

```
<text>
    <label>This is a label...</label>
</text>
```

5.3 <button> — Pushbutton

A clickable button. When clicked, its action chain runs. If no action is given, clicking the button exits the dialog and sets `EXIT` to the button's label.

Buttons can display a text label, an icon image, or both. Use `<label>` for the text and `<input type="file">` for the icon. If neither is given, the label defaults to '`OK`'.

Gtkdialog3 searches for icon files in the directories listed in the `GTKDIALOG_PIXMAP_PATH` environment variable if the file cannot be opened directly.

Tag attributes for button layout:

`image-position`
: Icon position relative to the label: `0` (left), `1` (right), `2` (top), `3` (bottom).

`spacing`
: Gap in pixels between icon and label (default 6).

`border-width`
: Inner padding between the button frame and its content.

From `examples/button/button_action_functions`:

```
<button>
    <label>exit</label>
    <action>echo You pressed the exit button</action>
    <action function="exit">Exit by button</action>
</button>
```

5.3.1 Pre-defined stock buttons

Gtkdialog3 provides shorthand stock buttons with a built-in label and icon:

- `<button stock="ok"/>`
- `<button stock="cancel"/>`
- `<button stock="help"/>`
- `<button stock="yes"/>`
- `<button stock="no"/>`

5.4 <entry> — Single-line text input

A single-line text field for entering strings. Actions fire when the user changes the content or presses Enter.

Use `<default>` to set the initial text. The `<sensitive>` sub-element accepts `enabled`, `disabled`, or `password`. In password mode the text is hidden with dots.

Exports the current text as the variable value.

From `examples/entry/entry`:

```
<entry>
    <default>Default entry text</default>
    <variable>ENTRY</variable>
</entry>
```

5.5 <checkbox> — Check mark

A labelled check box that the user can toggle on and off. Use `<default>` with `true` or `false` to set the initial state.

Actions fire when the state changes. Conditional actions with `if true` and `if false` prefixes let you run different commands depending on the new state.

Exports `true` or `false` as the variable value.

From `examples/checkbox/checkbox`:

```
<checkbox>
    <label>This is a checkbox...</label>
    <variable>CHECKBOX</variable>
    <action>echo Checkbox is $CHECKBOX now.</action>
    <action>if true enable:ENTRY</action>
    <action>if false disable:ENTRY</action>
</checkbox>
```

5.6 <radiobutton> — Mutually exclusive choice

A radio button that automatically groups with adjacent radio buttons. Only one button in a group can be active at a time — selecting one deselects the others.

Use `<default>` with `true` or `false` to set the initial state. Actions fire on the `toggled` signal.

Exports `true` or `false` as the variable value.

From `examples/radiobutton/radiobutton_attributes`:

```
<radiobutton active="true">
    <label>First choice</label>
    <variable>RADIOBUTTON1</variable>
    <action>echo RADIOBUTTON1 is $RADIOBUTTON1</action>
</radiobutton>
<radiobutton>
    <label>Second choice</label>
    <variable>RADIOBUTTON2</variable>
    <action>echo RADIOBUTTON2 is $RADIOBUTTON2</action>
</radiobutton>
```

5.7 <togglebutton> — Toggle button

A button that stays pressed when clicked and releases on the next click. Like regular buttons, toggle buttons can display a label, an icon, or both.

Use `<default>` with `true` or `false` to set the initial state. Actions fire on the `toggled` signal.

Exports `true` or `false` as the variable value.

From `examples/togglebutton/togglebutton_advanced`:

```
<togglebutton>
    <default>true</default>
    <label>Toggle</label>
    <variable>tgb0</variable>
    <action>save:tgb0</action>
    <output type="file">/tmp/outputfile</output>
</togglebutton>
```

5.8 `<switch>` — On/off switch

A sliding toggle switch, similar in function to a checkbox but with a modern visual appearance. Use `<default>` with `true` or `false` to set the initial state. Actions fire when the switch is toggled.

Exports `true` or `false` as the variable value.

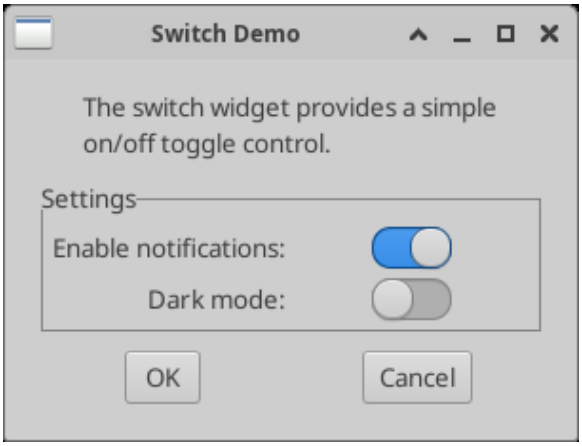

Figure 5.1: The switch widget from `examples/switch/switch`.

From `examples/switch/switch`:

```
<hbox homogeneous="true">
    <text space-expand="true" space-fill="true">
        <label>Enable notifications:</label>
    </text>
    <switch>
        <variable>NOTIFICATIONS</variable>
        <default>true</default>
    </switch>
</hbox>
```

5.9 <combobox> — Editable drop-down

A drop-down list with an editable text field. Items can be added with <item> sub-elements or loaded from a shell command with <input>. The user can select an existing item or type a new value.

Exports the current text (typed or selected) as the variable value.

From examples/combobox/combobox:

```
<combobox>
    <variable>COMBOBOX</variable>
    <item>First item</item>
    <item>Second item</item>
    <item>Third item</item>
</combobox>
```

5.10 <comboboxtext> — Fixed drop-down

A drop-down list where the user must choose from the available items (no free-form typing). Items can come from <item> elements, <input> commands, or <input type="file">. Use <default> to pre-select an item by its text.

Exports the selected item text as the variable value.

From examples/comboboxtext/comboboxtext_advanced:

```
<comboboxtext>
    <variable>cboComboBoxText0</variable>
    <item>First choice</item>
    <item>Second choice</item>
    <input>echo "This came from a shell command"</input>
    <action signal="changed">echo "Changed to $cboComboBoxText0"</action>
</comboboxtext>
```

5.11 <comboboxentry> — Drop-down with entry

A drop-down list combined with an editable entry field. This is a variant of <comboboxtext> that additionally allows the user to type arbitrary text. It supports the activate signal, which fires when the user presses Enter in the entry field.

Exports the current text as the variable value.

From examples/comboboxentry/comboboxentry_advanced:

```
<comboboxentry has-focus="true">
    <variable>cboComboBoxEntry3</variable>
    <default>Type something here...</default>
    <input type="file">/tmp/inoutfile</input>
    <output type="file">/tmp/inoutfile</output>
    <action signal="activate">save:cboComboBoxEntry3</action>
    <action signal="activate">refresh:cboComboBoxEntry3</action>
</comboboxentry>
```

5.12 `<list>` — Single-column list

A scrollable single-column list. Items can be added with `<item>` elements or loaded one-per-line from `<input>` or `<input type="file">`. Use the `selected-row` tag attribute to pre-select a row by index.

Actions fire when the selection changes.

Exports the text of the selected row as the variable value.

From `examples/list/list_attributes`:

```
<list selected-row="1">
    <variable>LIST</variable>
    <input>printf "First item\nSecond item\nThird item"</input>
    <action>echo "The chosen item is \"$LIST\""</action>
</list>
```

5.13 `<table>` — Multi-column table

A sortable multi-column table. Column headers are defined by a pipe-separated `<label>` and rows by pipe-separated `<item>` elements or pipe-separated lines from `<input>` / `<input file>`. Clicking a column header sorts the table by that column.

Key tag attributes:

`exported-column`
: Which column's value to export (default 0).

`selected-row`
: Pre-select a row by index.

`selection-mode`
: `single`, `multiple`, `browse`, or `none`.

`sort-function`
: Sort type: `0` (default), `1` (natural), `2` (natural, case-insensitive).

`column-visible`
: Pipe-separated `true`/`false` per column.

`column-header-active`
: Pipe-separated `true`/`false` to enable sorting per column.

Exports the selected cell text (from the exported column) as the variable value. With multiple selection, values are newline-separated.

From `examples/table/table`:

```
<table exported-column='2' selected-row='1'
    sort-function='1'
    column-header-active='true|true|true|false'>
    <variable>tbl0</variable>
    <label>Column 0  |Column 1  |Column 2  |Column 3</label>
    <input type="file">/tmp/inputfile</input>
    <item>1002|$RANDOM|0|$RANDOM</item>
    <action>echo "tbl0=$tbl0"</action>
</table>
```

5.14 <tree> — Multi-column tree view

A multi-column tree view, similar to <table> but predating it. Column headers are defined by a pipe-separated <label> and rows by pipe-separated <item> elements or input sources.

Exports the selected row as the variable value.

From examples/tree/tree_insert:

```
<tree>
    <label>Device    |Directory        |File</label>
    <item>Hard drive |/usr/            |letter.tex</item>
    <item>Hard drive |/etc/            |fstab</item>
    <item>Network    |alpha:/home      |quota.user</item>
    <variable>TREE</variable>
    <action>echo "TREE=$TREE"</action>
</tree>
```

5.15 <edit> — Multi-line text editor

A multi-line text editing area with scrollbars. Use <default> to set initial text, or <input type="file"> to load the contents of a file. The save: action writes the buffer to the file specified by <output type="file">.

Supports clipboard actions: cut:, copy:, paste:, and selectall: (see Section 7.9.20 [Clipboard actions], page 40).

Exports the full buffer text as the variable value.

From examples/edit/edit:

```
<edit>
    <variable>EDITOR</variable>
    <height>150</height>
    <width>350</width>
    <default>
"This is the default text of the editor."
    </default>
</edit>
```

5.16 <pixmap> — Static image

Displays a static image. The image file is specified with <input type="file">. Gtkdialog3 searches for the file in directories listed in the GTKDIALOG_PIXMAP_PATH environment variable if it cannot be opened directly.

Refreshing the widget reloads the image, which is useful for displaying dynamically generated graphics.

From examples/pixmap/pixmap:

```
<pixmap>
    <input type="file">help.png</input>
</pixmap>
```

5.17 <hscale>, <vscale> — Sliders

Horizontal and vertical slider controls for selecting a numeric value within a range. Use <item> elements to place custom marks along the scale.

Key tag attributes:

range-min
: Minimum value (default 0).

range-max
: Maximum value (default 100).

range-step
: Step increment (default 1).

range-value
: Initial value (default 0).

Actions fire when the value changes. Exports the current numeric value as the variable value.

From examples/hscale/hscale_advanced:

```
<hscale width-request="420" range-value="4">
    <variable>hscHScale0</variable>
    <action>echo "hscHScale0 changed to $hscHScale0"</action>
</hscale>
```

5.18 <spinbutton> — Numeric spinner

A text entry for numeric values with up/down arrow buttons. Supports the same range-min, range-max, range-step, and range-value tag attributes as scales.

Actions fire on value-changed (arrow clicks or typing) and activate (pressing Enter). Exports the current numeric value as the variable value.

From examples/spinbutton/spinbutton:

```
<spinbutton range-min="-50" range-max="50"
    range-step="5" range-value="10">
    <variable>SPB_RANGE</variable>
</spinbutton>
```

5.19 <colorbutton> — Colour picker

A button that displays a colour swatch and opens a colour chooser dialog when clicked. Use <default> to set the initial colour in #rrggbb format. If the use-alpha tag attribute is set to true, an alpha channel is also available and the format becomes #rrggbb|alpha.

Actions fire when a new colour is selected. Exports the colour string as the variable value.

From examples/colorbutton/colorbutton_advanced:

```
<colorbutton use-alpha="true">
    <default>#4488cc|32768</default>
    <variable>clb3</variable>
```

```
    <action>echo clb3 changed to $clb3</action>
</colorbutton>
```

5.20 <fontbutton> — Font picker

A button that displays a font name and opens a font chooser dialog when clicked. Use `<default>` or the `font-name` tag attribute to set the initial font using Pango syntax (e.g. '`Sans Bold 12`').

Actions fire when a new font is selected. Exports the Pango font description string as the variable value.

From `examples/fontbutton/fontbutton`:

```
<fontbutton font-name="Monospace 18" use-font="true">
    <variable>ftb1</variable>
    <action>echo ftb1 changed to $ftb1</action>
</fontbutton>
```

5.21 <progressbar> — Progress indicator

A horizontal progress bar. The `<input>` command runs in a background thread; each line of output is interpreted as either a number (0–100, setting the bar position) or text (updating the bar label). When the value reaches 100, the action chain fires.

Use `<label>` to set the initial bar text.

From `examples/progressbar/progressbar`:

```
<progressbar>
    <label>Some Text</label>
    <input>for i in $(seq 0 10 100); do
        echo $i; sleep 0.3; done</input>
    <action function="exit">Ready</action>
</progressbar>
```

5.22 <statusbar> — Status message bar

A message bar typically placed at the bottom of a window. The message can be set with `<label>`, `<default>`, `<input>`, or `<input type="file">`. Refreshing the widget re-evaluates its input source.

Exports the current message text as the variable value.

From `examples/statusbar/statusbar_advanced`:

```
<statusbar>
    <default>Ready</default>
    <variable>stb0</variable>
    <input>echo Status from a command</input>
    <output type="file">/tmp/outputfile</output>
</statusbar>
```

5.23 <timer> — Periodic timer

An invisible widget that fires its action chain at regular intervals. Use it to periodically refresh other widgets or run commands.

Key tag attributes:

`interval` The timer period (default 1).

`milliseconds`
: Set to `true` for millisecond precision; otherwise the interval is in seconds.

The timer runs while it is sensitive. Disabling the timer stops it; enabling it restarts it. Use `visible="false"` to hide the timer's internal label widget.

Exports `true` (running) or `false` (stopped) as the variable value.

From `examples/timer/timer_advanced`:

```
<timer milliseconds="true" interval="500"
    visible="false">
    <variable>tmr3</variable>
    <action>refresh:pix30</action>
    <action>refresh:pix31</action>
</timer>
```

5.24 <hseparator>, <vseparator> — Separators

Purely decorative horizontal or vertical lines used to visually separate groups of widgets. They have no value, input, or actions.

From `examples/hseparator/hseparator_advanced`:

```
<hseparator width-request="400"></hseparator>
```

5.25 <menubar> — Menu bar

A menu bar containing drop-down menus. Build the menu hierarchy with `<menu>` elements (each containing a `<label>` for the menu title) and `<menuitem>` elements inside. Use `<menuitemseparator/>` to insert separator lines between items.

Menu items support `<label>`, `<variable>`, and `<action>` sub-elements. A menuitem with `checkbox="true"` as a tag attribute becomes a check menu item that toggles on click.

From `examples/menubar/menubar`:

```
<menubar>
    <menu>
        <menuitem>
            <label>Open</label>
        </menuitem>
        <menuitemseparator></menuitemseparator>
        <menuitem>
            <label>Quit</label>
            <action>exit:quit</action>
        </menuitem>
        <label>File</label>
```

```
        </menu>
    </menubar>
```

5.26 <terminal> — Terminal emulator (optional)

An embedded VTE terminal emulator. Requires libvte (VTE 2.91) at build time. Use `gtkdialog3 --version` to check availability.

The terminal spawns a shell process (`/bin/sh` by default). Feed commands to it with `<input>` or `<input type="file">`. Actions fire on the `child-exited` signal when the shell exits.

Key tag attributes:

`argv0 ... argv127`
: Command-line arguments for the spawned process.

`text-foreground-color, text-background-color`
: Terminal colours.

`font-name`
: Terminal font (Pango syntax).

Exports the child process PID as the variable value.

The `set` action supports the following synthetic property for terminal widgets:

`command` Feed a command string to the terminal shell. A newline is appended automatically so the command executes immediately.

From `examples/standard/terminal`:

```
    <terminal argv0='/bin/bash' has-focus='true'>
        <variable>term</variable>
    </terminal>
    <button>
        <label>pwd</label>
        <action>set:term.command=pwd</action>
    </button>
```

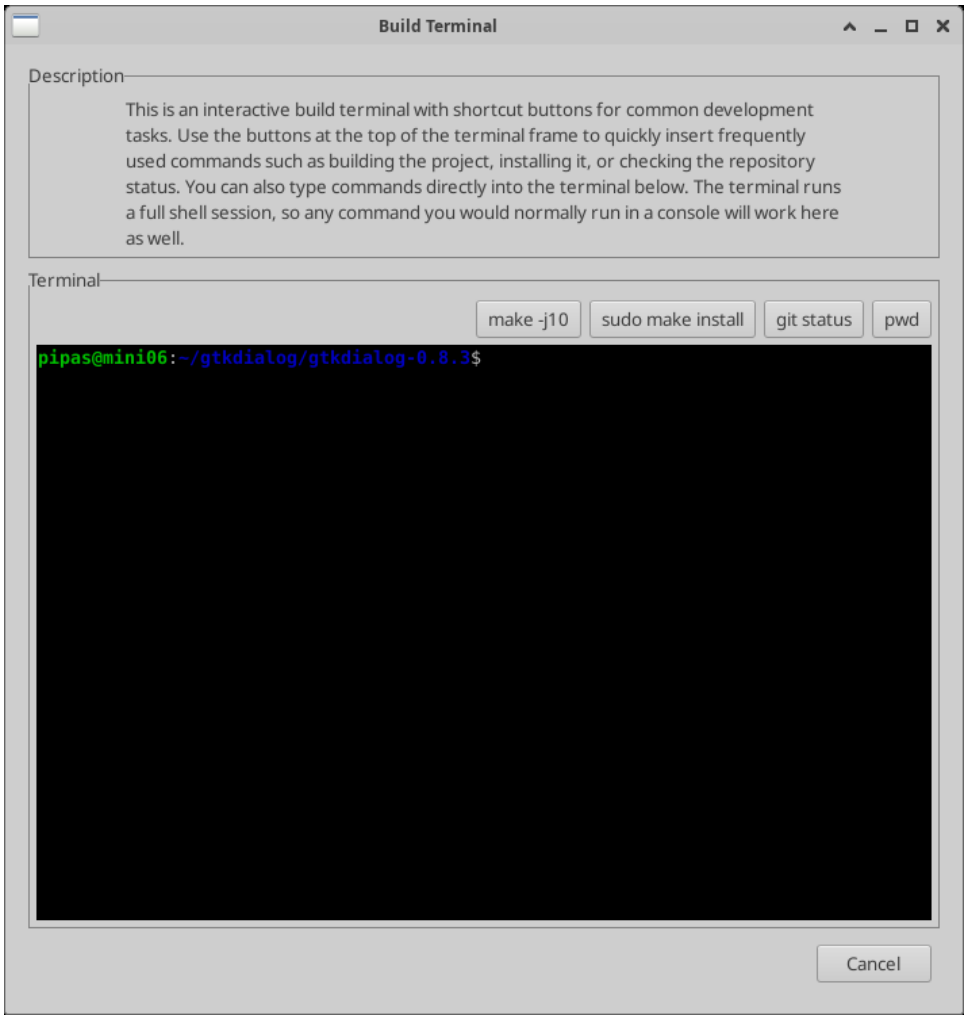

Figure 5.2: The terminal widget from `examples/standard/terminal`.

5.27 `<sourceview>` — Source editor (optional)

A text editor with syntax highlighting, line numbers, and auto-indentation. Requires libgtksourceview-4 at build time.

Use `<input type="file">` to load a source file. A companion `.lang` sidecar file (containing just the language ID) switches the highlighting language automatically on refresh.

Key tag attributes:

`language` Syntax highlighting language (e.g. `c`, `sh`, `python`, `json`).

`font-name`
: Font name and size (e.g. '`Monospace 12`').

`show-line-numbers`
: Show line numbers (`true`/`false`, default `true`).

`highlight-current-line`
: Highlight the current line (`true`/`false`, default `true`).

`tab-width`
: Tab width in characters (default 4).

Exports the full buffer text as the variable value.

The `set` action supports the following synthetic properties for sourceview widgets:

`file` Load the contents of the named file into the editor buffer.

`language` Switch the syntax highlighting language (e.g. `c`, `sh`, `json`).

From `examples/standard/sourceview`:

```
<sourceview language='c' font-name='Monospace 12'>
    <variable>srcEditor</variable>
    <input type='file'>/tmp/hello.c</input>
</sourceview>
<button>
    <label>Bash Script</label>
    <action>set:srcEditor.file=/tmp/greet.sh</action>
    <action>set:srcEditor.language=sh</action>
</button>
```

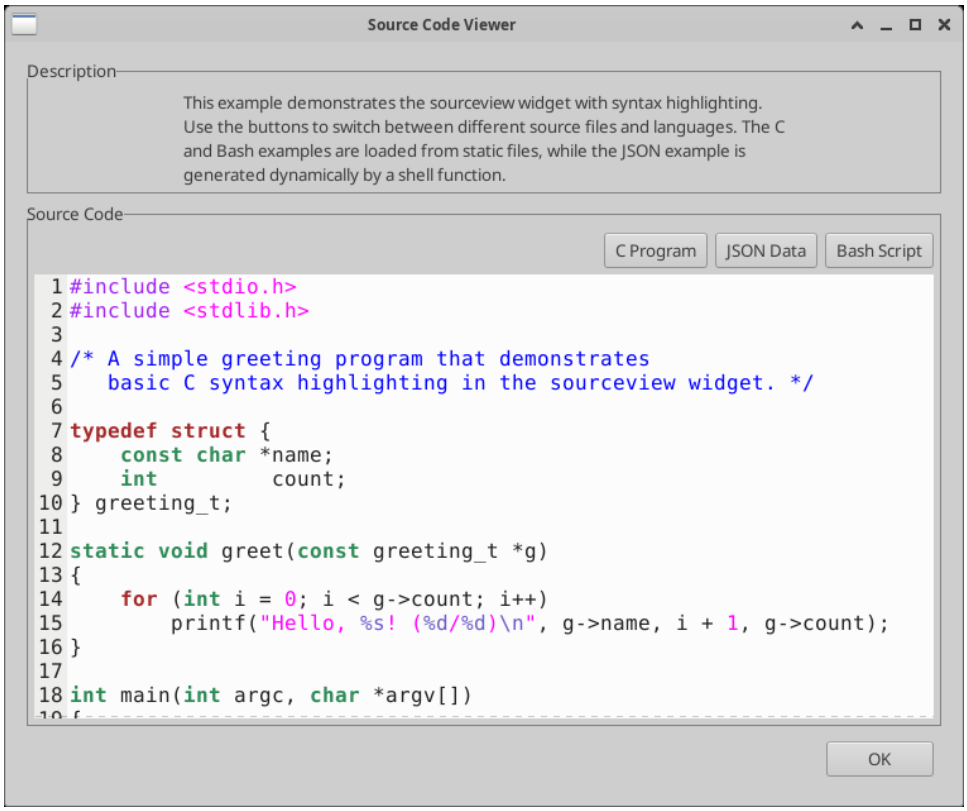

Figure 5.3: The sourceview widget from `examples/standard/sourceview`.

5.28 `<webview>` — Web browser (optional)

An embedded web browser using WebKitGTK. Requires libwebkit2gtk at build time.

Use `<default>` to set the initial URL, or `<input type="file">` to load content dynamically. The input file may contain a URL (starting with `http://` or `https://`), a local file path (starting with `/`), or raw HTML.

Exports the current URI as the variable value.

The `set` action supports the following synthetic property for webview widgets:

`uri` Navigate to the given URL or local `file://` path.

From `examples/standard/webview`:

```
<webview space-expand='true' space-fill='true'>
    <variable>webPage</variable>
    <default>file:///tmp/home.html</default>
</webview>
<button>
    <label>LWN.net</label>
    <action>set:webPage.uri=https://lwn.net</action>
</button>
```

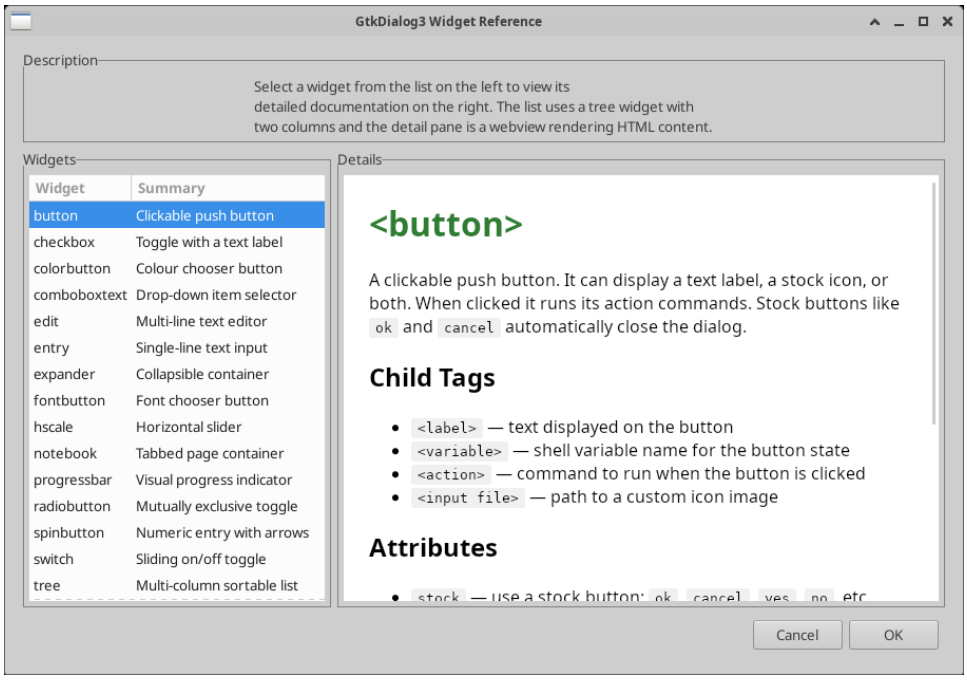

Figure 5.4: The webview widget from `examples/standard/list`, showing a tree-driven widget reference with HTML detail pane.

6 Containers

Containers are layout widgets that hold other widgets and arrange them on screen. Every widget must be placed inside a container.

6.1 <vbox>, <hbox> — Box layouts

The most common containers. A <vbox> stacks its children vertically; an <hbox> arranges them horizontally. Boxes can be nested to create complex layouts.

Key tag attributes:

`spacing` Pixel gap between children (default 5).

`homogeneous`
: Give all children equal size (`true`/`false`).

`space-expand`
: Default expand packing for children (`true`/`false`).

`space-fill`
: Default fill packing for children (`true`/`false`).

`border-width`
: Padding inside the container.

From `examples/vbox/vbox`:

```
<vbox>
    <text><label>First</label></text>
    <text><label>Second</label></text>
    <hbox>
        <button stock="ok"/>
        <button stock="cancel"/>
    </hbox>
</vbox>
```

6.2 <frame> — Labelled frame

A container with a visible border and a title label. Use the `title` tag attribute to set the frame label. The frame label can also be set dynamically via <input> or <input type="file">.

From `examples/frame/frame`:

```
<frame title="Frame1">
    <text><label>Label1</label></text>
    <entry></entry>
</frame>
```

6.3 <notebook> — Tabbed pages

A container with tabbed pages. Each direct child becomes a page. Tab labels are set with the `labels` tag attribute (pipe-separated), or auto-generated as '`Page 1`', '`Page 2`', etc.

Exports the current page index (starting at 0) as the variable value.

From `examples/notebook/notebook`:

```
<notebook labels="Checkbox|Radiobutton|Tree">
    <vbox>
        <checkbox><label>A checkbox</label></checkbox>
    </vbox>
    <vbox>
        <radiobutton><label>First</label></radiobutton>
        <radiobutton><label>Second</label></radiobutton>
    </vbox>
    <vbox>
        <text><label>Third page</label></text>
    </vbox>
</notebook>
```

6.4 <eventbox> — Event container

An invisible container that adds mouse and keyboard event handling to its single child widget. Useful for detecting clicks, mouse movement, or key presses on widgets that do not normally emit those signals.

From `examples/eventbox/eventbox`:

```
<eventbox above-child="false" visible-window="true">
    <text><label>Click me</label></text>
    <action signal="button-press-event">
        echo "Clicked!"</action>
    <action signal="enter-notify-event">
        echo "Mouse entered"</action>
</eventbox>
```

6.5 <expander> — Collapsible container

A container with a clickable header that expands or collapses to show or hide its children. Use the `expanded` tag attribute to set the initial state (`true` or `false`).

Actions fire when the expander is toggled. Exports `true` (expanded) or `false` (collapsed) as the variable value.

From `examples/expander/expander`:

```
<expander expanded="true">
    <vbox>
        <entry><default>entry</default></entry>
    </vbox>
    <label>Details</label>
    <variable>exp0</variable>
```

```
    <action>echo "exp0=$exp0"</action>
</expander>
```

6.6 <buttonbox> — Button layout

A container that arranges buttons with consistent spacing. Use it instead of `<hbox>` for dialog button rows to get standard platform-appropriate layout.

Key tag attributes:

`layout` Button arrangement: `spread`, `edge`, `start`, `end` (default), or `center`.

`orientation`
: `horizontal` (default) or `vertical`.

From `examples/hbox/hbox`:

```
<buttonbox layout='end'>
    <button stock='cancel'/>
    <button stock='help'/>
</buttonbox>
```

6.7 <window> — Top-level window

The outermost container. Every dialog has at least one window. Use the `title` tag attribute to set the title bar text, and `icon-name` to set the title bar icon.

Multiple windows can be opened using the `launch:` action (see Section 7.9.3 [launch], page 36).

From `examples/window/window_attributes`:

```
<window title="Example Window"
    icon-name="gtk-dialog-warning">
    <vbox>
        <text><label>Hello, World!</label></text>
        <hbox>
            <button stock="ok"/>
            <button stock="cancel"/>
        </hbox>
    </vbox>
</window>
```

7 Widget Sub-elements

Widgets are configured by placing sub-elements between their opening and closing tags. These sub-elements control the widget's label, initial value, data source, output destination, variable name, and behaviour when the user interacts with it.

This chapter documents every sub-element. Not every widget supports every sub-element — see the individual widget descriptions (see Chapter 5 [Widgets], page 15) for which ones apply to each widget type.

7.1 <variable>

Names the widget so its value can be read back by the calling script. When the dialog closes, all named variables are printed to standard output as shell assignments (see Section 4.3 [Capturing output], page 7).

```
<entry>
    <variable>USERNAME</variable>
</entry>
```

On exit this produces `USERNAME="whatever the user typed"`.

If a widget has no `<variable>` element, gtkdialog3 assigns an internal name automatically, but auto-named variables are not exported.

The optional tag attribute `export="false"` keeps the variable internal — it can still be referenced by actions such as `refresh:` or `enable:`, but it will not appear in the output.

7.2 <label>

Sets the display text of the widget. The meaning depends on the widget type:

- `<text>` — the label text shown on screen.
- `<button>` — the button caption.
- `<checkbox>`, `<radiobutton>` — the text beside the check mark.
- `<frame>` — the frame title.
- `<window>` — the window title bar text (alternatively use the `title` tag attribute).
- `<tree>`, `<table>` — pipe-separated column headers.
- `<progressbar>` — the text displayed inside the bar.
- `<statusbar>` — the initial status message.

```
<text>
    <label>Please enter your name:</label>
</text>
```

7.3 <default>

Sets the initial value of the widget before any user interaction. The meaning depends on the widget type:

- `<entry>` — pre-fills the text field.
- `<checkbox>`, `<radiobutton>`, `<togglebutton>`, `<switch>` — `true` or `false` sets the toggle state.

- `<comboboxtext>`, `<comboboxentry>` — selects the matching item.
- `<hscale>`, `<vscale>`, `<spinbutton>` — sets the initial numeric value.
- `<colorbutton>` — sets the initial colour in `#rrggbb` format.
- `<fontbutton>` — sets the initial font in Pango syntax.
- `<webview>` — the initial URL to load.
- `<statusbar>` — the initial status message.

```
<entry>
    <default>localhost</default>
    <variable>HOSTNAME</variable>
</entry>
```

7.4 <input>

Loads data into the widget from an external source. There are three forms:

7.4.1 Shell command

`<input>`*command*`</input>` runs the command in a subshell and uses its standard output to populate the widget. For entry and text widgets, the output becomes the displayed text. For list, tree, and table widgets, each line of output becomes a row. For combobox widgets, each line becomes an item.

```
<list>
    <input>ls /usr/bin</input>
    <variable>FILES</variable>
</list>
```

7.4.2 File

`<input type="file">`*path*`</input>` reads the contents of the named file. The file is also monitored for changes — if it is modified on disk, a `file-changed` signal is emitted, which can trigger an action to refresh the widget automatically.

```
<terminal argv0='/bin/bash' has-focus='true'>
    <variable>term</variable>
    <input type="file">$TMPDIR/cmd</input>
</terminal>
```

7.4.3 Stock and theme icons

`<input type="stock">`*icon-id*`</input>` loads a GTK stock icon by ID. `<input type="icon">`*icon-name*`</input>` loads a theme icon by name. These forms are used primarily with button and pixmap widgets.

```
<button>
    <input type="stock">gtk-open</input>
    <label>Open</label>
</button>
```

When the `refresh:` action is used on a widget, its `<input>` is re-evaluated and the widget is updated with the new data.

7.5 <output>

Specifies a file where the widget's value will be written when the `save:` action is triggered.

```
<edit>
    <variable>EDITOR</variable>
    <input type="file">/tmp/myfile.txt</input>
    <output type="file">/tmp/myfile.txt</output>
</edit>
<button>
    <label>Save</label>
    <action function="save">EDITOR</action>
</button>
```

Works on most widget types including entries, text editors, checkboxes, comboboxes, scales, and spinbuttons.

7.6 <item>

Adds an entry to list-type and menu widgets. Multiple `<item>` elements can appear in a single widget. The meaning depends on the widget type:

- `<comboboxtext>`, `<comboboxentry>`, `<combobox>`, `<list>` — each item becomes a selectable option.
- `<tree>`, `<table>` — each item becomes a row, with columns separated by the pipe character `|`.
- `<hscale>`, `<vscale>` — each item defines a scale mark at a position.

Items in tree and table widgets can have icons via tag attributes: `<item stock="icon-name">` or `<item icon="icon-name">`.

```
<comboboxtext>
    <item>First choice</item>
    <item>Second choice</item>
    <item>Third choice</item>
    <variable>CHOICE</variable>
</comboboxtext>
```

7.7 <sensitive>

Controls whether the widget is interactive. A widget that is not sensitive is greyed out and does not respond to mouse or keyboard input.

Accepted values: `true`, `false`, `enabled`, `disabled`, `yes`, `no`, `0`.

The special value `password` (entry widgets only) makes the widget sensitive but hides the text with dots, as is common for password fields.

The legacy tag name `<visible>` is an alias for `<sensitive>` and works identically.

```
<entry>
    <default>secret</default>
    <sensitive>password</sensitive>
    <variable>PASSWORD</variable>
```

```
</entry>
```

Widgets can also be enabled and disabled at runtime using the `enable:` and `disable:` actions (see Section 7.9.6 [enable], page 37, see Section 7.9.7 [disable], page 37).

7.8 <width> and <height>

Set the minimum size of the widget in pixels. For terminal widgets, the values are in character columns and rows instead of pixels.

```
<edit>
    <width>400</width>
    <height>300</height>
    <variable>EDITOR</variable>
</edit>
```

These can also be set using the `width-request` and `height-request` tag attributes on the opening tag, which is the standard GTK property approach:

```
<edit width-request="400" height-request="300">
    <variable>EDITOR</variable>
</edit>
```

7.9 <action>

Defines what happens when the user interacts with the widget. Every interactive widget can have one or more `<action>` elements. Gtkdialog3 executes them in the order they appear, so you can write a sequence of instructions that runs as a small program.

Actions can be written in two equivalent syntaxes:

Colon syntax: `<action>`*type*`:`*argument*`</action>`
Attribute syntax: `<action function="`*type*`">`*argument*`</action>`

The colon syntax is shorter; the attribute syntax allows additional tag attributes such as `signal` (which GTK signal triggers the action) and `condition` (a condition that must be true for the action to run). Both forms are case-insensitive in the action type.

The available action types are listed in the following subsections.

7.9.1 Shell commands

If no action type is given, the action text is executed as a shell command. Gtkdialog3 starts the user's `$SHELL` (falling back to `/bin/sh`) to run the command. Before the command runs:

1. All widget variables are exported as environment variables so the command can read the current state of the dialog.
2. If gtkdialog3 was started with the `-i` *file* option, that file is sourced by the subshell first, making its functions available to the command.
3. The command runs and gtkdialog3 waits for it to finish. Append `&` to the command to run it in the background without waiting.

From `examples/button/button_action_functions`:

```
<button>
```

```
    <label>exit</label>
    <action>echo You pressed the exit button</action>
    <action function="exit">Exit by button</action>
</button>
```

7.9.2 exit:*Value*

Closes the dialog immediately. All widget variables are printed to standard output, followed by `EXIT="Value"`. The calling script can evaluate this output to find out which button or action caused the dialog to close.

From `examples/button/button_action_functions`:

```
<button>
    <label>exit</label>
    <action>echo You pressed the exit button</action>
    <action function="exit">Exit by button</action>
</button>
```

7.9.3 launch:*Name*

Opens a new window whose definition is stored in the environment variable *Name*. If a window with that variable name already exists, it is brought to the front instead of creating a duplicate. The target window must be defined in a separate exported shell variable containing a `<window>` element.

From `examples/languages/bash_script_multi_window`:

```
<button>
    <label>launch winLaunch1</label>
    <action function="launch">winLaunch1</action>
</button>
```

7.9.4 closewindow:*Name*

Closes the window identified by *Name*. All variables belonging to that window are removed. If other windows remain open, the program continues running; if this was the last window, the program exits and prints all remaining variables.

From `examples/languages/bash_script_multi_window`:

```
<button>
    <label>closewindow winLaunch1</label>
    <action function="closewindow">winLaunch1</action>
</button>
```

7.9.5 presentwindow:*Name*

Brings the named window to the front and gives it keyboard focus. Unlike `launch:`, this does not create a new window — it only raises an existing one. Useful when you have multiple windows open and want to direct the user's attention to a specific one.

From `examples/miscellaneous/presentwindow`:

```
<button>
    <label>presentwindow winLaunch1</label>
```

```
    <action function="presentwindow">winLaunch1</action>
</button>
```

7.9.6 enable:*Name*

Makes the named widget interactive again after it has been disabled. If the widget is already enabled, nothing happens. Works on any widget type.

From `examples/button/button_action_functions`:

```
<button>
    <label>enable</label>
    <action function="enable">ENTRY</action>
    <action function="enable">CHECKBOX</action>
</button>
```

7.9.7 disable:*Name*

Greys out the named widget and prevents all user interaction with it. If the widget is already disabled, nothing happens. Works on any widget type.

From `examples/button/button_action_functions`:

```
<button>
    <label>disable</label>
    <action function="disable">ENTRY</action>
    <action function="disable">CHECKBOX</action>
</button>
```

7.9.8 show:*Name*

Makes the named widget visible. If the widget is inside a scrolled container, the container is also shown so the widget actually appears on screen.

Typically used together with `hide:` and conditional prefixes to toggle parts of the interface:

From `examples/miscellaneous/show_and_hide`:

```
<togglebutton active="true">
    <label>button</label>
    <action>if true show:btnExample</action>
    <action>if false hide:btnExample</action>
</togglebutton>
```

7.9.9 hide:*Name*

Makes the named widget invisible. The widget still exists and keeps its value, but it is removed from the visible layout. If the widget is inside a scrolled container, the container is also hidden.

From `examples/miscellaneous/show_and_hide`:

```
<menuitem checkbox="true" label="vbox Left">
    <action>if true show:vbxLeft</action>
    <action>if false hide:vbxLeft</action>
</menuitem>
```

7.9.10 activate:*Name*

Simulates the default activation of the named widget — clicking a button, toggling a checkbox, pressing Enter in an entry, and so on. This triggers the widget's own action chain as if the user had interacted with it directly.

From `examples/miscellaneous/activate`:

```
<button>
    <label>Activate everything else</label>
    <action function="activate">button</action>
    <action function="activate">checkbox</action>
    <action function="activate">entry</action>
    <action function="activate">togglebutton</action>
</button>
```

7.9.11 grabfocus:*Name*

Gives keyboard focus to the named widget. After this action, keyboard input goes to the target widget. Useful for directing the user to a specific entry field or text editor after an event.

From `examples/miscellaneous/grabfocus`:

```
<button>
    <label>entry</label>
    <action>grabfocus:entEntry</action>
</button>
```

7.9.12 refresh:*Name*

Reloads the content of the named widget from its data source. If the widget has an `<input>` or `<input type="file">` directive, it is re-evaluated and the widget is updated with the new data. All widget variables are exported before the refresh, so input commands can access the current state of the dialog.

From `examples/checkbox/checkbox_conditional_refreshing`:

```
<checkbox sensitive="false">
    <variable>chkCheck</variable>
    <input type="file">/tmp/inputdir/chkCheck</input>
    <action>refresh:txt0</action>
    <action>refresh:txt1</action>
</checkbox>
```

7.9.13 save:*Name*

Writes the current value of the named widget to the file specified by its `<output type="file">` attribute. Works on most widget types including entries, text editors, checkboxes, comboboxes, scales, and spinbuttons.

From `examples/edit/edit_actions`:

```
<button>
    <label>Save</label>
    <action function="save">EDITOR</action>
</button>
```

7.9.14 `fileselect:`*Name*

Opens a file chooser dialog and places the selected path into the named widget (typically an entry). The file chooser's behaviour is controlled by tag attributes on the target widget:

`fs-action`
: The type of selection: `file`, `newfile`, `folder`, or `newfolder`.

`fs-title`
: The title of the file chooser dialog.

`fs-folder`
: The initial directory shown.

`fs-filters`
: Pipe-separated file glob patterns, e.g. `*.txt|*.html`.

`fs-filters-mime`
: Pipe-separated MIME types, e.g. `text/plain|text/html`.

From `examples/miscellaneous/fileselect_advanced`:

```
<entry fs-action="file" fs-folder="/usr/share/doc"
    fs-filters="*.txt|*.html"
    fs-title="Select an existing file">
    <variable>ent1</variable>
</entry>
<button>
    <input type="stock">gtk-new</input>
    <action>fileselect:ent1</action>
</button>
```

7.9.15 `clear:`*Name*

Removes all content from the named widget. For text editors and entries this clears the text; for lists, trees, and tables this removes all rows; for comboboxes this removes all items.

From `examples/edit/edit_actions`:

```
<button>
    <label>Clear</label>
    <action function="clear">EDITOR</action>
</button>
```

7.9.16 `removeselected:`*Name*

Removes the currently selected item from the named widget. Most commonly used with list, tree, and table widgets to let the user delete individual rows. For text editors, it removes the selected text range.

From `examples/tree/tree_insert`:

```
<button>
    <input type="stock">gtk-remove</input>
    <action function="removeselected">TREE</action>
</button>
```

7.9.17 `break`

Stops executing the remaining actions in the current action chain. Actions listed before the `break` run normally; actions listed after it are skipped. Typically used with a `condition` attribute to conditionally stop the chain.

From `examples/miscellaneous/break`:

```
<button label="Break">
    <action>echo This action is above the break</action>
    <action condition="active_is_true(CHECKBOX)"
            function="break">""</action>
    <action>echo This action is below the break</action>
</button>
```

7.9.18 `insert:`*Source*`,`*Target*

Takes the value from the *Source* widget and inserts it into the *Target* widget. For list and tree widgets, a new row is added. For entry widgets, the text is set. For edit widgets, the text is inserted at the cursor position.

From `examples/tree/tree_insert`:

```
<entry>
    <variable>ENTRY</variable>
</entry>
<button>
    <input type="stock">gtk-add</input>
    <action function="insert">ENTRY,TREE</action>
</button>
```

7.9.19 `append:`*Source*`,`*Target*

Works the same way as `insert:` — takes the value from *Source* and adds it to *Target*. Both `insert:` and `append:` add new rows to list and tree widgets and set text in entry and edit widgets.

7.9.20 Clipboard actions

The following four actions operate on text editor (`<edit>`) widgets and use the system clipboard.

7.9.20.1 `cut:`*Name*

Cuts the selected text from the named editor to the clipboard.

7.9.20.2 `copy:`*Name*

Copies the selected text from the named editor to the clipboard without removing it.

7.9.20.3 `paste:`*Name*

Pastes the clipboard contents into the named editor at the cursor position.

7.9.20.4 `selectall:`*Name*

Selects all text in the named editor.

From `examples/edit/edit_actions`:

```
<button>
    <label>Select All</label>
    <action function="selectall">EDITOR</action>
</button>
<button>
    <label>Cut</label>
    <action function="cut">EDITOR</action>
</button>
<button>
    <label>Copy</label>
    <action function="copy">EDITOR</action>
</button>
<button>
    <label>Paste</label>
    <action function="paste">EDITOR</action>
</button>
```

7.9.21 `set:`*`Widget.Property=Value`*

Sets a GTK property on the named widget at runtime. The argument is parsed as *Widget*`.`*Property*`=`*Value*, where *Widget* is the variable name, *Property* is any GTK property that the widget supports, and *Value* is the new value. The same properties that work as tag attributes at widget creation time (e.g. `title`, `sensitive`, `label`) can be set with this action.

Set the window title:

```
<action>set:myWindow.title=Processing...</action>
```

Disable a widget:

```
<action>set:myButton.sensitive=false</action>
```

Using attribute syntax:

```
<action type='set'>myButton.sensitive=false</action>
```

This action can replace the common pattern of writing to a temporary file and then refreshing a widget. For example, loading a URL in a webview:

Before (two actions + temp file):

```
<action>echo https://lwn.net > /tmp/nav</action>
<action>refresh:webPage</action>
```

After (one action, no temp file):

```
<action>set:webPage.uri=https://lwn.net</action>
```